

CRYPTOGRAPHY & NETWORK SECURITY

Secure File Vault – AES-256 File Encryption and Decryption System Using Python

1. Introduction

Data stored on computers may contain personal, academic, financial, or other sensitive information. Unauthorized access to such files can lead to data leakage or misuse.

The **Secure File Vault** is a Python-based application that protects files using **AES-256 encryption**. The user provides a password, which is converted into a secure encryption key using PBKDF2 and SHA-256. The application provides a simple graphical interface for encrypting and decrypting files.

2. Problem Statement

Normal files are stored in readable form and can be accessed if an unauthorized person gets access to the computer. There is a need for a simple system that can securely encrypt files and allow only authorized users with the correct password to recover the original data.

3. Objectives

- To develop a secure file encryption and decryption system.
- To use AES-256 for protecting file data.
- To generate a secure key from the user's password.
- To detect incorrect passwords and modified encrypted files.
- To provide a simple and user-friendly graphical interface.
- To prevent accidental overwriting of decrypted files.

4. Python Libraries Used

Library	Purpose
<code>tkinter</code>	Creates the graphical user interface
<code>Crypto.Cipher</code>	Performs AES encryption and decryption
<code>Crypto.Protocol.KDF</code>	Generates encryption key using PBKDF2
<code>Crypto.Hash</code>	Provides SHA-256
<code>Crypto.Random</code>	Generates random salt
<code>os</code>	Handles file names and paths

5. Technologies Used

- **Programming Language:** Python
- **GUI:** Tkinter
- **IDE:** Visual Studio Code
- **Encryption Library:** PyCryptodome
- **Operating System:** Windows

6. System / Project Design

The system follows two main processes:

Encryption Process:

```
Select File
      ↓
Enter Password
      ↓
Generate Random Salt
      ↓
PBKDF2 + SHA-256
      ↓
Generate AES-256 Key
      ↓
AES-EAX Encryption
      ↓
Encrypted .enc File
```

Decryption Process:

```
Select .enc File
      ↓
Enter Password
      ↓
Read Salt + Nonce + Tag
      ↓
Generate AES-256 Key
      ↓
AES-EAX Verification
      ↓
Decrypt File
      ↓
Original File
```

7. Cryptography / Network Security Concepts

AES-256:

AES is a symmetric encryption algorithm used to protect data. AES-256 uses a 256-bit key and provides strong protection for sensitive files.

PBKDF2:

PBKDF2 converts the user's password into a strong cryptographic key using a salt and repeated processing.

SHA-256:

SHA-256 is used as the hash function during password-based key generation.

Random Salt:

A random salt makes the generated key different even when the same password is used.

AES-EAX:

AES-EAX provides encryption along with authentication. The authentication tag helps detect incorrect passwords or modified files.

8. Methodology / Procedure

1. Start the Secure File Vault application.
2. Select the file using the **Browse** button.
3. Enter a password of at least 8 characters.
4. The application checks the password strength.
5. A random 16-byte salt is generated.
6. PBKDF2 with SHA-256 generates a 32-byte AES-256 key.
7. AES-EAX encrypts the file data.
8. Salt, nonce, authentication tag and ciphertext are stored in the `.enc` file.
9. For decryption, the encrypted file and password are selected.
10. The application regenerates the AES key using the stored salt.
11. AES-EAX verifies the authentication tag.
12. If verification succeeds, the original file is recovered.
13. If the password is wrong or the file has been modified, decryption fails.

9. Results / Output

The developed application was tested with different conditions.

Test Results:

Test	Result
File Encryption	Successful
Correct Password Decryption	Successful
Wrong Password	Detected
Weak Password	Detected
Password Strength	Working
File Tampering	Detected
Existing Decrypted File	Protected from Overwriting